

ENTERPRISE AGENTIC RAG

SYSTEM FLOW & CLOUD REFERENCE HANDBOOK

A Comprehensive Guide to Architecture, Setup, and Observability

This handbook contains the architectural blueprint, data flows, cloud configuration schemas, and runtime operations playbook for the Enterprise RAG system. It serves as a recovery guide and architectural guide for engineering teams managing these services.

Stack Overview:

FastAPI - Streamlit - LangGraph - NeMo Guardrails - Portkey Gateway
Qdrant Cloud - Google Gemini Embeddings - Groq LLM - Pydantic Logfire - LangSmith

1. Executive Summary & Architecture

This Enterprise RAG (Retrieval-Augmented Generation) application represents a highly robust, production-ready pattern that combines advanced orchestration, input/output filtering, and multi-cloud integrations. Unlike simple RAG templates, this system includes enterprise guardrails (NeMo), cognitive state management (LangGraph), semantic reranking (FlashRank), unified LLM API governance (Portkey), and end-to-end telemetry (Logfire + LangSmith).

The Architectural Gate System

To ensure production stability, the backend implements a two-stage gate check for every incoming message:

1. NeMo Guardrails Gate (Stage 1): Acts as a firewall. Checks for off-topic queries, jailbreaks, and pre-defined conversational flows (such as greeting and capabilities) before invoking any agentic graph. If a guardrail fires, the request skips the pipeline and returns a secure canned response instantly.
2. LangGraph Execution Engine (Stage 2): Processes valid technical inquiries using a compiled state graph equipped with conversational memory (MemorySaver) for thread-based contextual memory.

2. Universal Ingestion Pipeline

The Ingestion Pipeline parses heterogeneous data sources, converts them to high-density vector representations, and builds a persistent index in Qdrant Cloud. It is invoked via: `python -m app.ingestion.processor DATA --wipe`

Ingestion Phase Breakdown:

1. Directory Scanning & Parsing: Scans the DATA folder. Files are automatically routed to custom extractors:
 - PDF: Parsed locally using 'pypdf' or 'pdfplumber'.
 - HTML: Structured using 'BeautifulSoup4'.
 - Word/PowerPoint: Split using 'python-docx' and 'python-pptx'.
2. Adaptive Chunking: Chunks text using a Recursive Character Text Splitter (target size 1,000 chars, 200 char overlap).
3. Local Cache Persistence: Saves raw and parsed chunk metadata locally under `processed_data/` for audit trails.
4. Dual Embedding Strategy: Computes vector embeddings using 'models/gemini-embedding-2-preview' (3,072 dimensions) via Google AI Studio. If the Gemini API is unreachable, it automatically falls back to local SentenceTransformers using 'all-mpnet-base-v2' (768 dimensions).
5. Qdrant Upsert: Writes the vectors and payload (text, source filename, source type) into Qdrant Cloud.

3. Query Routing & Agentic Execution Flow

When a query passes the Guardrails Gate, it is handed over to the compiled LangGraph Agent state machine. The state graph manages transitions and tracks the conversation context dynamically.

LangGraph Node Operations

1. Planner Node (`planner_node`):

Uses Llama 3.3 70B via the LLM client to evaluate the entire thread history + latest user message. It determines if the request is 'CONVERSATIONAL' (greetings, simple follow-ups like 'what did I say?') or a technical inquiry requiring vector search.

2. Router Decision:

- If 'CONVERSATIONAL', routes the state directly to the Responder Node (skips vector database entirely).
- If technical, the refined search query is output, and it routes to the Retriever Node.

3. Retriever Node (`retrieve_node`):

- Queries Qdrant Cloud using the modern 'query_points' API for the top 15 most similar chunks.
- Semantic Reranking: Standard vector cosine similarity can miss contextual details. The retriever passes all 15 chunks to a local FlashRank Ranker using a quantized Cross-Encoder (ms-marco-MiniLM-L-6-v2) to compute semantic scores. It retains the top 5 most relevant documents.

4. Responder Node (`generate_node`):

Pulls conversation memory plus the retrieved context (if technical) and constructs a structured prompt. It invokes the Groq LLM through the Portkey Unified API Gateway.

Conversational Memory & State Tracking

Conversational memory is backed by a local MemorySaver checkpoint. It uses a 'thread_id' to store and append message histories in the state variables. State fields include: messages (appended via `operator.add`), `current_query`, `documents`, `plan`, `status`, and `final_answer`.

4. Cloud Integrations & Credentials Reference

The application integrates six distinct SaaS/PaaS systems via standard environment variables defined in `.env`. Here is the credential mapping schema:

Service Name	Env Variable	Purpose & Description
Qdrant Cloud	QDRANT_API_KEY QDRANT_CLUSTER_ENDPOINT	Hosts the vector database for RAG document retrieval.
Google AI Studio	GEMINI_API_KEY	Powers the high-density 3,072-dimension embeddings.
Groq Cloud	GROQ_API_KEY GROQ_FALLBACK_API_KEY	Runs Llama-3.3-70b-versatile and Llama-3.1-8b-instant models.
Portkey Gateway	PORTKEY_API_KEY PORTKEY_CONFIG_ID	AI gateway handling prompt cache status, retries, and fallbacks.
Pydantic Logfire	LOGFIRE_TOKEN	Centralized logging and APM trace logs for fast debugging.
LangSmith	LANGSMITH_API_KEY LANGSMITH_TRACING=true	Traces LLM prompts, inputs, and outputs across LangChain/LangGraph.

5. Observability, Telemetry & Tracing

Pydantic Logfire Integration

Logfire instruments the FastAPI app, incoming HTTP requests, Qdrant db operations, and semantic search queries. If `LOGFIRE_TOKEN` is present in `.env`, telemetry is uploaded directly. If empty, the backend runs in a safe offline mode using `logfire.configure(send_to_logfire=False)` to avoid boot blocks in non-interactive terminals.

LangSmith & Portkey Logging

Langsmith tracks LangChain & LangGraph agents at runtime. Separately, Portkey handles caching. The `generate_node` parses `x-portkey-cache-status` headers: if standard requests hit the Portkey cache, the cache hit is recorded, saving money and reducing API latency to millisecond thresholds.

6. Operations Playbook & Recovery

Follow these standard terminal run commands to spin up the services and run ingestion:

1. Ingestion Execution

```
# Wipe database and load fresh enterprise data
python -m app.ingestion.processor DATA --wipe

# Process specific folder only without wiping
python -m app.ingestion.processor DATA/true_data true
```

2. Start API Backend Service

```
# Runs FastAPI backend on port 8000
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

3. Start Streamlit UI Dashboard

```
# Runs frontend client dashboard on port 8501
python -m streamlit run ui/app.py
```

7. Quick Diagnostic Rules

Use this guide to solve recurring startup errors quickly:

1. Port Conflict (Errno 10048): If port 8000 is occupied, kill the process:
 - Windows PowerShell: `Stop-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess -Force`
2. Settings Import Issue: If 'ImportError: cannot import name settings' occurs, verify `config.py` instantiates `settings = Settings()` (all lowercase).
3. Missing Portkey Client Fallback: If Portkey is disabled (no API key), ensure fallback works. The wrapper in `app/gateway/client.py` automatically injects `model=settings.GROQ_MODEL` into OpenAI calls.
4. Streamlit Connection Failed: If UI shows 'query: no scheme supplied', ensure `BACKEND_URL` in `.env` is not blank, or that the UI correctly defaults to `http://127.0.0.1:8000`.